

DAY 3: Videos and watch progress, FRONTEND

12 new files + 3 small edits. Three screens.

Route	Who	What
/manage-videos	ADMIN, INSTRUCTOR	pick a course (an instructor sees only their own), upload (title, description, MP4/WebM file) with a progress bar and Cancel , a table of the videos with Started / Finished / Average columns, Watch and Delete
/watch/:courseId	CUSTOMER (bought), ADMIN, INSTRUCTOR	the player with a lesson list. For customers it resumes ("Continuing from 0:05"), saves progress, shows a Completed badge and moves to the next lesson at the end
/my-learning	CUSTOMER	each bought course that has videos: "1 of 2 lessons finished", a percent bar and Start / Continue / Watch again

Details worth knowing:

- The upload page reads the video's length in the browser and sends it, so percentages are right. It refuses non-MP4/WebM files and files over 200 MB before uploading, and shows the server's own message if the server says no (for example 413).
- Progress is saved every 10 s while playing, on pause, on changing lesson and on leaving the page.
- The player link is a short-lived ticket. If it expires while someone is watching, the page asks for a new one **once** and continues from the same second; if that also fails, it shows a clear message.
- Admins and instructors can preview lessons; no progress is stored for them.

Honest status

Built and tested in a real browser with a **real playable WebM file** against a pretend backend following the same API: **48 checks, 0 failures** (upload, bad file refused, server 413 message, delete with confirmation, instructor sees only own course, non-buyer blocked, play, pause saves progress, **resume after reload at the same second**, 90% completes, going back keeps completed, switching lessons, my-learning percentages, expired-ticket handling, admin statistics, role guards, phone width, no JS errors). Not tested against the real Spring backend, and MP4 playback depends on the codec in your file.

The production bundle grows to about 1.04 MB (Angular prints a *warning* for the 1 MB budget; the error limit is 2 MB, so the build still succeeds).

Edits to existing files

1. **constant.ts**: inside `API: VIDEO: '/api/video'`,

2. **app.module.ts**: import `VideoModule` and put it **before** `AppRoutingModule`:

```
import { VideoModule } from './modules/video/video.module';
// imports: [... , VerificationModule, InstructorModule, VideoModule, AppRoutingModule]
```

3. `components/navbar/navbar.component.html`: add a **Videos** link for ADMIN and INSTRUCTOR and **My Learning** for CUSTOMER, in the desktop `<nav>` and the `<mat-menu>`:

```
<!-- ADMIN block and INSTRUCTOR block (desktop) -->
<a routerLink="/manage-videos" routerLinkActive="active">Videos</a>
<!-- CUSTOMER block, after My Enrollments (desktop) -->
<a routerLink="/my-learning" routerLinkActive="active">My Learning</a>

<!-- the same three, inside <mat-menu> -->
<a mat-menu-item routerLink="/manage-videos">Videos</a>
<a mat-menu-item routerLink="/my-learning">My Learning</a>
```

Check it (with the Day 3 backend)

1. Admin: **Videos**, choose a course, pick a small MP4, press Upload: the bar fills and "Video uploaded." shows. Pick a `.txt`: "Choose an MP4 or WebM video."
2. A customer who bought that course: **My Learning** shows the course. Press Start, watch 10 seconds, close the tab.
3. Open My Learning again, press **Continue**: "Continuing from 0:10".
4. Drag to the end: the lesson shows **Completed** and the next lesson starts.
5. Admin, **Videos**, same course: Started 1, Finished 1, Average 100%.
6. A customer who did **not** buy the course opening `/watch/<id>` sees "Enroll in this course to watch its videos".
7. Instructor: sees only their own courses in the list.

Troubleshooting

Symptom	Cause
<code>/watch/1</code> shows the home page	<code>VideoModule</code> is after <code>AppRoutingModule</code>
the video shows but never plays	codec; re-export as H.264 + AAC MP4 or VP8/VP9 WebM
progress is never saved	the user is not a CUSTOMER, or the backend <code>PUT /progress</code> rule is missing
upload stops at 100% and fails	the server's multipart limit is too low (backend edit 4b)

Complete code

`models/video.model.ts` (new file)

```
/** Shapes of the video API (see VideoController). */

export interface VideoProgress {
  positionSec: number;
  percent: number;
  completed: boolean;
}

export interface CourseVideo {
  id: number;
  courseId: number;
  title: string;
```

```

description: string | null;
sizeBytes: number;
durationSec: number;
uploadedAtMs: number;
progress?: VideoProgress | null;
}

export interface VideoTicket {
  ticket: string;
  url: string;
  expiresInSeconds: number;
}

export interface VideoStat {
  videoId: number;
  title: string;
  durationSec: number;
  viewers: number;
  completedCount: number;
  averagePercent: number;
}

export interface CourseVideoStats {
  courseId: number;
  enrolledCustomers: number;
  videos: VideoStat[];
}

export interface LearningItem {
  courseId: number;
  courseType: string;
  totalVideos: number;
  completedVideos: number;
  percent: number;
  resumeVideoId: number | null;
  resumePositionSec: number;
}

/** "75" -> "1:15", "3725" -> "1:02:05", 0 -> "--". */
export function formatDuration(seconds: number): string {
  if (!seconds || seconds < 0) { return '--'; }
  const s = Math.floor(seconds % 60);
  const m = Math.floor((seconds / 60) % 60);
  const h = Math.floor(seconds / 3600);
  const two = (n: number) => (n < 10 ? '0' + n : String(n));
  return h > 0 ? `${h}:${two(m)}:${two(s)}` : `${m}:${two(s)}`;
}

/** 1536 -> "1.5 KB", 5242880 -> "5.0 MB". */
export function formatSize(bytes: number): string {
  if (bytes < 1024) { return bytes + ' B'; }
  if (bytes < 1024 * 1024) { return (bytes / 1024).toFixed(1) + ' KB'; }
  if (bytes < 1024 * 1024 * 1024) { return (bytes / 1024 / 1024).toFixed(1) + ' MB'; }
  return (bytes / 1024 / 1024 / 1024).toFixed(2) + ' GB';
}

```

services/video.service.ts (new file)

```

import { HttpClient, HttpEvent } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API, API_URL } from '../constant';
import { CourseVideo, CourseVideoStats, LearningItem, VideoProgress, VideoTicket } from
'../models/video.model';

/** Lesson videos: upload, list, watch (ticket + progress) and the staff statistics. */
@Injectable({
  providedIn: 'root'
})

```

```

export class VideoService {
  public apiUrl = API_URL;

  constructor(private http: HttpClient) {}

  /** Returns the raw events so the page can draw an upload progress bar. */
  upload(courseId: number, form: FormData): Observable<HttpEvent<CourseVideo>> {
    return this.http.post<CourseVideo>(`${this.apiUrl}${API.VIDEO}/course/${courseId}`, form, {
      reportProgress: true, observe: 'events' });
  }

  list(courseId: number): Observable<CourseVideo[]> {
    return this.http.get<CourseVideo[]>(`${this.apiUrl}${API.VIDEO}/course/${courseId}`);
  }

  stats(courseId: number): Observable<CourseVideoStats> {
    return this.http.get<CourseVideoStats>(`${this.apiUrl}${API.VIDEO}/course/${courseId}/stats`);
  }

  myLearning(): Observable<LearningItem[]> {
    return this.http.get<LearningItem[]>(`${this.apiUrl}${API.VIDEO}/my-learning`);
  }

  ticket(videoId: number): Observable<VideoTicket> {
    return this.http.get<VideoTicket>(`${this.apiUrl}${API.VIDEO}/${videoId}/ticket`);
  }

  saveProgress(videoId: number, positionSec: number, durationSec: number): Observable<VideoProgress> {
    return this.http.put<VideoProgress>(`${this.apiUrl}${API.VIDEO}/${videoId}/progress`, { positionSec,
      durationSec });
  }

  delete(videoId: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}${API.VIDEO}/${videoId}`);
  }

  /** The player's address: the ticket URL from the server, placed after the API address. */
  streamUrl(ticket: VideoTicket): string {
    return `${this.apiUrl}${ticket.url}`;
  }
}

```

modules/video/video.module.ts (new file)

```

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { SharedModule } from '../../../shared/shared.module';
import { AuthGuard } from '../../../components/authguard/authguard.guard';
import { ROLES } from '../../../constant';
import { INSTRUCTOR_ROLE } from '../../../models/instructor.model';
import { ManageVideosComponent } from '../../../components/manage-videos/manage-videos.component';
import { WatchVideoComponent } from '../../../components/watch-video/watch-video.component';
import { MyLearningComponent } from '../../../components/my-learning/my-learning.component';

const routes: Routes = [
  { path: 'manage-videos', component: ManageVideosComponent, canActivate: [AuthGuard], data: { roles:
    [ROLES.ADMIN, INSTRUCTOR_ROLE] } },
  { path: 'my-learning', component: MyLearningComponent, canActivate: [AuthGuard], data: { roles:
    [ROLES.CUSTOMER] } },
  { path: 'watch/:courseId', component: WatchVideoComponent, canActivate: [AuthGuard], data: { roles:
    [ROLES.ADMIN, ROLES.CUSTOMER, INSTRUCTOR_ROLE] } }
];

/**
 * Lesson videos for admins, instructors and customers.
 * Import it in AppModule BEFORE AppRoutingModule so these routes are registered before the "" fallback.
 */
@NgModule({
  declarations: [ManageVideosComponent, WatchVideoComponent, MyLearningComponent],

```

```

    imports: [SharedModule, RouterModule.forChild(routes)]
  })
  export class VideoModule {}

```

components/manage-videos/manage-videos.component.ts (new file)

```

import { HttpEventType } from '@angular/common/http';
import { Component, OnDestroy, OnInit } from '@angular/core';
import { MatDialog } from '@angular/material/dialog';
import { ActivatedRoute } from '@angular/router';
import { forkJoin, of, Subscription } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { ConfirmDialogComponent } from '../confirm-dialog/confirm-dialog.component';
import { Course } from '../../models/course.model';
import { CourseVideo, CourseVideoStats, VideoStat, formatDuration, formatSize } from
  '../../models/video.model';
import { AuthService } from '../../services/auth.service';
import { CourseService } from '../../services/course.service';
import { InstructorService } from '../../services/instructor.service';
import { VideoService } from '../../services/video.service';
import { INSTRUCTOR_ROLE } from '../../models/instructor.model';

/**
 * For the admin and for approved instructors: pick a course, upload videos to it, see them and see how far
the
 * customers have watched. An instructor only sees their own courses; the server enforces the same rule.
 */
@Component({
  selector: 'app-manage-videos',
  templateUrl: './manage-videos.component.html',
  styleUrls: ['./manage-videos.component.css']
})
export class ManageVideosComponent implements OnInit, OnDestroy {
  /** Hint for the person uploading; the server has the real limit (video.max-size-mb). */
  readonly maxMb = 200;
  readonly fmtDuration = formatDuration;
  readonly fmtSize = formatSize;

  courses: Course[] = [];
  coursesLoaded = false;
  notApproved = false;
  courseId: number | null = null;

  videos: CourseVideo[] = [];
  stats: CourseVideoStats | null = null;
  videosLoading = false;
  loadError = '';

  title = '';
  description = '';
  file: File | null = null;
  fileDuration = 0;
  fileError = '';
  uploading = false;
  uploadPercent = 0;
  uploadError = '';
  message = '';
  private uploadSub?: Subscription;

  constructor(private auth: AuthService, private courseApi: CourseService, private instructors:
InstructorService,
    private videoApi: VideoService, private dialog: MatDialog, private route: ActivatedRoute) {}

  ngOnInit(): void {
    const wanted = Number(this.route.snapshot.queryParamMap.get('courseId'));
    const source = this.auth.getRole() === INSTRUCTOR_ROLE ? this.instructors.myCourses() :
this.courseApi.viewAllCourses();
    source.subscribe({
      next: list => {

```

```

        this.courses = list;
        this.coursesLoaded = true;
        if (wanted && list.some(c => c.courseId === wanted)) { this.courseId = wanted; this.loadCourse(); }
    },
    error: err => {
        this.coursesLoaded = true;
        if (err.status === 403) { this.notApproved = true; } else { this.loadError = 'Could not load your
courses. Please try again.'; }
    }
});
}

ngOnDestroy(): void {
    this.uploadSub?.unsubscribe();
}

onCourseChange(): void {
    this.message = '';
    this.uploadError = '';
    this.resetForm();
    this.loadCourse();
}

loadCourse(): void {
    if (this.courseId === null) { this.videos = []; this.stats = null; return; }
    const id = this.courseId;
    this.videosLoading = true;
    this.loadError = '';
    forkJoin({
        videos: this.videoApi.list(id),
        stats: this.videoApi.stats(id).pipe(catchError(() => of(null)))
    }).subscribe({
        next: r => {
            if (this.courseId !== id) { return; }
            this.videos = r.videos;
            this.stats = r.stats;
            this.videosLoading = false;
        },
        error: err => {
            this.videosLoading = false;
            this.loadError = err.status === 403 ? 'You cannot manage videos of this course.' : 'Could not load
the videos. Please try again.';
        }
    });
}

statFor(videoId: number): VideoStat | undefined {
    return this.stats?.videos.find(s => s.videoId === videoId);
}

onFileChosen(event: Event): void {
    const input = event.target as HTMLInputElement;
    this.file = null;
    this.fileDuration = 0;
    this.fileError = '';
    const f = input.files && input.files.length ? input.files[0] : null;
    if (!f) { return; }
    if (!/\. (mp4|webm)$/i.test(f.name)) { this.fileError = 'Choose an MP4 or WebM video.'; input.value =
''; return; }
    if (f.size > this.maxMb * 1024 * 1024) { this.fileError = `This video is larger than ${this.maxMb}
MB.`; input.value = ''; return; }
    this.file = f;
    if (!this.title.trim()) { this.title = f.name.replace(/\.([^.] +$/ , '').replace(/[_-]+/g, ' ').slice(0,
150); }
    this.readDuration(f);
}

/** Asks the browser for the length of the chosen file, so the progress percentage can be calculated
later. */
private readDuration(f: File): void {
    const url = URL.createObjectURL(f);

```

```

    const probe = document.createElement('video');
    probe.preload = 'metadata';
    const done = () => { URL.revokeObjectURL(url); probe.removeAttribute('src'); };
    probe.onloadedmetadata = () => { this.fileDuration = isFinite(probe.duration) ?
Math.round(probe.duration) : 0; done(); };
    probe.onerror = () => { this.fileDuration = 0; done(); };
    probe.src = url;
  }

  get canUpload(): boolean {
    return this.courseId !== null && !!this.file && this.title.trim().length >= 2 && !this.uploading;
  }

  upload(): void {
    if (!this.canUpload || this.courseId === null || !this.file) { return; }
    const form = new FormData();
    form.append('title', this.title.trim());
    if (this.description.trim()) { form.append('description', this.description.trim()); }
    form.append('durationSec', String(this.fileDuration));
    form.append('file', this.file);
    this.uploading = true;
    this.uploadPercent = 0;
    this.uploadError = '';
    this.message = '';
    this.uploadSub = this.videoApi.upload(this.courseId, form).subscribe({
      next: ev => {
        if (ev.type === HttpEventType.UploadProgress && ev.total) {
          this.uploadPercent = Math.min(99, Math.round((100 * ev.loaded) / ev.total));
        } else if (ev.type === HttpEventType.Response) {
          this.uploading = false;
          this.uploadPercent = 100;
          this.message = 'Video uploaded.';
          this.resetForm();
          this.loadCourse();
        }
      },
      error: err => {
        this.uploading = false;
        this.uploadError = err.status === 413 ? (err.error?.message || 'The video is too large.')
          : [400, 403, 404, 429].includes(err.status) ? (err.error?.message || 'The video could not be
uploaded.')
          : 'Could not upload the video. Please try again.';
      }
    });
  }

  cancelUpload(): void {
    this.uploadSub?.unsubscribe();
    this.uploading = false;
    this.uploadPercent = 0;
    this.uploadError = 'Upload cancelled.';
  }

  askDelete(v: CourseVideo): void {
    this.dialog.open(ConfirmDialogComponent, {
      data: { title: 'Delete video?', message: 'Delete "' + v.title + '"? The file and the customers\'
watch progress for it are removed.', confirmText: 'Yes, delete', cancelText: 'Cancel', icon: 'delete',
danger: true }
    }).afterClosed().subscribe(yes => {
      if (!yes) { return; }
      this.videoApi.delete(v.id).subscribe({
        next: () => { this.message = 'Video deleted.'; this.loadCourse(); },
        error: err => { this.message = ''; this.loadError = err.error?.message || 'The video could not be
deleted.'; }
      });
    });
  }

  private resetForm(): void {
    this.title = '';
    this.description = '';
  }

```

```

    this.file = null;
    this.fileDuration = 0;
    this.fileError = '';
    const input = document.getElementById('videoFile') as HTMLInputElement | null;
    if (input) { input.value = ''; }
  }

  trackById(_: number, v: CourseVideo): number { return v.id; }
}

```

components/manage-videos/manage-videos.component.html (new file)

```

<div class="page">
  <div class="page-head">
    <div>
      <h2>Course videos</h2>
      <p>Upload lessons to a course and see how far your customers have watched.</p>
    </div>
  </div>

  <div class="form-note" id="notApproved" *ngIf="notApproved">
    Your instructor account is not approved yet, so you cannot manage videos.
    <a routerLink="/instructor-register">See your application</a>
  </div>
  <div class="form-error" role="alert" id="loadError" *ngIf="loadError"><mat-icon>error_outline</mat-icon>
  {{ loadError }}</div>
  <p class="form-note ok" id="videoMessage" *ngIf="message">{{ message }}</p>

  <div class="picker" *ngIf="!notApproved">
    <label for="courseSelect">Course</label>
    <select id="courseSelect" class="native" [(ngModel)]="courseId" (ngModelChange)="onCourseChange()"
  [disabled]="uploading">
      <option [ngValue]="null">Choose a course</option>
      <option *ngFor="let c of courses" [ngValue]="c.courseId">{{ c.courseType }}</option>
    </select>
    <p class="empty small" id="noCourses" *ngIf="coursesLoaded && !courses.length">
      There are no courses yet. Add a course first, then come back to upload videos.
    </p>
  </div>

  <ng-container *ngIf="courseId !== null && !notApproved">
    <section class="panel" id="uploadPanel">
      <h3>Upload a video</h3>
      <div class="grid2">
        <div class="field">
          <label for="videoTitle">Title</label>
          <input id="videoTitle" class="native" maxlength="150" [(ngModel)]="title" [disabled]="uploading"
        />
        </div>
        <div class="field">
          <label for="videoFile">Video file (MP4 or WebM, up to {{ maxMb }} MB)</label>
          <input id="videoFile" class="native file" type="file" accept="video/mp4,video/webm,.mp4,.webm"
        (change)="onFileChosen($event)" [disabled]="uploading" />
        </div>
        <div class="field full">
          <label for="videoDescription">Description (optional)</label>
          <textarea id="videoDescription" class="native" rows="2" maxlength="500" [(ngModel)]="description"
        [disabled]="uploading"></textarea>
        </div>
      </div>
      <p class="muted" id="fileInfo" *ngIf="file">{{ file.name }} &middot; {{ fmtSize(file.size) }}<span
      *ngIf="fileDuration"> &middot; {{ fmtDuration(fileDuration) }}</span></p>
      <div class="form-error small" role="alert" id="fileError" *ngIf="fileError"><mat-
      icon>error_outline</mat-icon>{{ fileError }}</div>
      <div class="form-error" role="alert" id="uploadError" *ngIf="uploadError"><mat-
      icon>error_outline</mat-icon>{{ uploadError }}</div>

      <div class="bar" id="uploadProgress" *ngIf="uploading" role="progressbar" [attr.aria-
      valuenow]="uploadPercent" aria-valuemin="0" aria-valuemax="100">

```

```

        <div class="fill" [style.width.%]="uploadPercent"></div>
        <span>{{ uploadPercent }}%</span>
    </div>

    <div class="actions">
        <button mat-flat-button color="primary" class="solid" id="uploadVideo" type="button"
[disabled]="!canUpload" (click)="upload()">
            <mat-icon>upload</mat-icon>{{ uploading ? 'Uploading...' : 'Upload video' }}
        </button>
        <button mat-stroked-button class="ghost" id="cancelUpload" type="button" *ngIf="uploading"
(click)="cancelUpload()">Cancel</button>
    </div>
</section>

<section class="panel">
    <div class="panel-head">
        <h3>Videos in this course</h3>
        <span class="chip" id="enrolledCount" *ngIf="stats">{{ stats.enrolledCustomers }} enrolled</span>
    </div>
    <p class="empty" id="noVideos" *ngIf="!videosLoading && !loadError && !videos.length">No videos yet.
Upload the first lesson above.</p>

    <div class="table" *ngIf="videos.length">
        <div class="row head">
            <span>Lesson</span><span>Length</span><span>Size</span><span>Started</span><span>Finished</span><span>Average</span><span></span>
        </div>
        <div class="row" *ngFor="let v of videos; let i = index; trackBy: trackById" [attr.id]="`video` +
v.id">
            <span class="name"><strong>{{ i + 1 }}. {{ v.title }}</strong><small *ngIf="v.description">{{
v.description }}</small></span>
            <span data-label="Length">{{ fmtDuration(v.durationSec) }}</span>
            <span data-label="Size">{{ fmtSize(v.sizeBytes) }}</span>
            <span data-label="Started" class="viewers">{{ statFor(v.id)?.viewers ?? 0 }}</span>
            <span data-label="Finished" class="finished">{{ statFor(v.id)?.completedCount ?? 0 }}</span>
            <span data-label="Average" class="average">{{ statFor(v.id)?.averagePercent ?? 0 }}%</span>
            <span class="row-actions">
                <a mat-stroked-button class="ghost watch" [routerLink]="['/watch', v.courseId]" [queryParams]="
{ video: v.id }">Watch</a>
                <button mat-stroked-button class="ghost danger del" type="button"
(click)="askDelete(v)">Delete</button>
            </span>
        </div>
    </div>
</section>
</ng-container>
</div>

```

components/manage-videos/manage-videos.component.css (new file)

```

/* Admin / instructor: course videos (Midnight) */
.picker { display: flex; flex-direction: column; gap: 6px; max-width: 460px; margin-bottom: 22px; }
label { font: 700 13px var(--display); color: var(--gold-text); }
.native { width: 100%; padding: 11px 12px; border-radius: 10px; border: 1px solid var(--line); background:
var(--surface-2); color: var(--ink); font: 500 15px Inter, sans-serif; }
.native:focus { outline: 2px solid var(--gold); outline-offset: 1px; }
.native.file { padding: 8px 10px; }
textarea.native { resize: vertical; }
.panel { margin-bottom: 22px; padding: 20px; border-radius: var(--radius); background: var(--surface);
border: 1px solid var(--line); box-shadow: var(--shadow); }
.panel h3 { font-size: 19px; margin: 0 0 14px; }
.panel-head { display: flex; align-items: center; justify-content: space-between; gap: 10px; margin-bottom:
10px; }
.panel-head h3 { margin: 0; }
.grid2 { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; margin-bottom: 10px; }
.field { display: flex; flex-direction: column; gap: 6px; min-width: 0; }
.field.full { grid-column: 1 / -1; }
.muted { color: var(--muted); font-size: 13.5px; margin: 0 0 10px; overflow-wrap: anywhere; }
.actions { display: flex; flex-wrap: wrap; gap: 10px; margin-top: 8px; }

```

```

.bar { position: relative; height: 24px; border-radius: 999px; background: var(--surface-2); border: 1px
solid var(--line); overflow: hidden; margin: 6px 0 4px; }
.bar .fill { height: 100%; background: var(--gold); transition: width .2s ease; }
.bar span { position: absolute; inset: 0; display: grid; place-items: center; font: 700 12.5px Inter, sans-
serif; color: #fff; mix-blend-mode: difference; }
.chip { padding: 3px 12px; border-radius: 999px; background: var(--gold-soft); color: var(--gold-text);
font: 700 12.5px Inter, sans-serif; }
.empty { padding: 28px 0; text-align: center; color: var(--muted); }
.empty.small { padding: 6px 0; text-align: left; }
.table { display: flex; flex-direction: column; }
.row { display: grid; grid-template-columns: minmax(0, 3fr) .8fr .9fr .8fr .8fr .8fr auto; gap: 12px;
align-items: center; padding: 12px 4px; border-top: 1px solid var(--line); color: var(--ink-2); font-size:
14.5px; }
.row.head { border-top: 0; font: 700 12px var(--display); color: var(--gold-text); }
.name { display: flex; flex-direction: column; gap: 2px; min-width: 0; overflow-wrap: anywhere; }
.name strong { color: var(--ink); }
.name small { color: var(--muted); font-size: 13px; }
.row-actions { display: flex; gap: 8px; justify-content: flex-end; }
.danger { color: var(--danger); border-color: rgba(248, 113, 113, .5); }
@media (max-width: 760px) {
  .grid2 { grid-template-columns: 1fr; }
  .row.head { display: none; }
  .row { grid-template-columns: 1fr 1fr; padding: 14px 4px; }
  .row .name, .row .row-actions { grid-column: 1 / -1; }
  .row [data-label]::before { content: attr(data-label) ": "; color: var(--muted); }
  .row-actions { justify-content: flex-start; }
}

```

components/watch-video/watch-video.component.ts (new file)

```

import { Component, ElementRef, OnDestroy, OnInit, ViewChild } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';
import { VideoService } from '../../services/video.service';
import { CourseVideo, formatDuration } from '../../models/video.model';
import { ROLES } from '../../constant';

/**
 * The lesson player. A customer's position is saved every few seconds, when they pause, switch lesson or
 * leave,
 * so the next visit continues where they stopped. Admins and instructors can watch too, but have no
 * progress.
 */
@Component({
  selector: 'app-watch-video',
  templateUrl: './watch-video.component.html',
  styleUrls: ['./watch-video.component.css']
})
export class WatchVideoComponent implements OnInit, OnDestroy {
  readonly fmtDuration = formatDuration;
  /** How often the position is saved while the video plays, in milliseconds. */
  private readonly saveEveryMs = 10000;

  courseId = 0;
  videos: CourseVideo[] = [];
  current: CourseVideo | null = null;
  src = '';
  loading = true;
  error = '';
  playError = '';
  resumedAt = 0;
  isCustomer = false;

  private player?: ElementRef<HTMLVideoElement>;
  private timer: any = null;
  private startAt = 0;
  private retried = false;

  @ViewChild('player') set playerRef(ref: ElementRef<HTMLVideoElement> | undefined) { this.player = ref; }

```

```

    constructor(private route: ActivatedRoute, private router: Router, private api: VideoService, private
auth: AuthService) {}

    ngOnInit(): void {
        this.isCustomer = this.auth.getRole() === ROLES.CUSTOMER;
        this.courseId = Number(this.route.snapshot.paramMap.get('courseId'));
        const wanted = Number(this.route.snapshot.queryParamMap.get('video'));
        this.api.list(this.courseId).subscribe({
            next: list => {
                this.videos = list;
                this.loading = false;
                const first = list.find(v => v.id === wanted) || list.find(v => !v.progress?.completed) || list[0];
                if (first) { this.open(first, false); }
            },
            error: err => {
                this.loading = false;
                this.error = err.status === 403 ? (err.error?.message || 'Enroll in this course to watch its
videos.')
: err.status === 404 ? 'This course was not found.' : 'Could not load the videos. Please try
again.';
            }
        });
    }

    ngOnDestroy(): void {
        this.saveNow();
        this.stopTimer();
    }

    open(v: CourseVideo, saveFirst = true): void {
        if (saveFirst) { this.saveNow(); }
        this.stopTimer();
        this.current = v;
        this.playError = '';
        this.retried = false;
        this.resumedAt = 0;
        const p = v.progress;
        this.startAt = p && !p.completed && p.positionSec > 3 ? p.positionSec : 0;
        this.loadSource(v);
    }

    private loadSource(v: CourseVideo): void {
        this.src = '';
        this.api.ticket(v.id).subscribe({
            next: t => { if (this.current?.id === v.id) { this.src = this.api.streamUrl(t); } },
            error: err => { this.playError = err.error?.message || 'This video could not be opened.'; }
        });
    }

    // ----- player events
    onMetadata(): void {
        const el = this.player?.nativeElement;
        if (!el || !this.current) { return; }
        if (this.startAt > 0 && this.startAt < (el.duration || Infinity)) {
            el.currentTime = this.startAt;
            this.resumedAt = this.startAt;
        }
    }

    onPlay(): void {
        if (!this.isCustomer || this.timer) { return; }
        this.timer = setInterval(() => this.saveNow(), this.saveEveryMs);
    }

    onPause(): void {
        this.stopTimer();
        this.saveNow();
    }

    onEnded(): void {

```

```

    this.stopTimer();
    this.saveNow();
    const i = this.videos.findIndex(v => v.id === this.current?.id);
    if (i >= 0 && i < this.videos.length - 1) { this.open(this.videos[i + 1], false); }
  }

  /** The link may have run out (tickets are short-lived): ask for a new one once and carry on from the
  same second. */
  onError(): void {
    const el = this.player?.nativeElement;
    if (!this.current || !this.src) { return; }
    if (!this.retried) {
      this.retried = true;
      this.startAt = el ? Math.floor(el.currentTime) : 0;
      this.loadSource(this.current);
    } else {
      this.playError = 'This video cannot be played. It may be missing or in a format your browser does not
support.';
    }
  }
}

  /** Saves the position (customers only). Failures are ignored on purpose: the next save will try again.
  */
  saveNow(): void {
    const el = this.player?.nativeElement;
    const v = this.current;
    if (!this.isCustomer || !el || !v || !this.src) { return; }
    const pos = Math.floor(el.currentTime || 0);
    const dur = Math.floor(isFinite(el.duration) ? el.duration : 0);
    if (pos <= 0 && !v.progress) { return; }
    this.api.saveProgress(v.id, pos, dur).subscribe({
      next: p => { v.progress = p; },
      error: () => { /* try again at the next save */ }
    });
  }

  private stopTimer(): void {
    if (this.timer) { clearInterval(this.timer); this.timer = null; }
  }

  back(): void {
    this.router.navigate([this.isCustomer ? '/my-learning' : '/manage-videos'], this.isCustomer ? {} : {
queryParams: { courseId: this.courseId } });
  }

  trackById(_ : number, v: CourseVideo): number { return v.id; }
}

```

components/watch-video/watch-video.component.html (new file)

```

<div class="page watch">
  <div class="page-head">
    <div>
      <h2>Course lessons</h2>
      <p *ngIf="isCustomer">Your place is saved automatically, so you can continue any time.</p>
      <p *ngIf="!isCustomer">Preview of the lessons as customers see them. Watch progress is only kept for
customers.</p>
    </div>
    <button mat-stroked-button class="ghost" type="button" id="backButton" (click)="back()">{{ isCustomer ?
'My learning' : 'Manage videos' }}</button>
  </div>

  <p class="empty" *ngIf="loading">Loading lessons...</p>
  <div class="form-error" role="alert" id="watchError" *ngIf="error"><mat-icon>error_outline</mat-icon>{{
error }}</div>
  <p class="empty" id="noLessons" *ngIf="!loading && !error && !videos.length">No lessons have been added
to this course yet.</p>

  <div class="layout" *ngIf="videos.length">

```

```

<section class="stage">
  <div class="screen">
    <video #player id="player" *ngIf="src" [src]="src" controls controlsList="nodownload" playsinline
preload="metadata"
      (loadedmetadata)="onMetadata()" (play)="onPlay()" (pause)="onPause()" (ended)="onEnded()"
(error)="onError()"></video>
    <div class="loading-screen" *ngIf="!src && !playError">Opening the lesson...</div>
    <div class="loading-screen bad" id="playError" *ngIf="playError">{{ playError }}</div>
  </div>
  <div class="info" *ngIf="current">
    <h3 id="currentTitle">{{ current.title }}</h3>
    <p class="muted" *ngIf="current.description">{{ current.description }}</p>
    <p class="resume" id="resumeNote" *ngIf="resumedAt > 0">Continuing from {{ fmtDuration(resumedAt)
}}</p>
    <span class="done" id="completedBadge" *ngIf="isCustomer && current.progress?.completed"><mat-
icon>check_circle</mat-icon>Completed</span>
  </div>
</section>

<aside class="list" aria-label="Lessons">
  <h3>Lessons</h3>
  <ol>
    <li *ngFor="let v of videos; let i = index; trackBy: trackById" [attr.id]='lesson' + v.id"
[class.on]="current?.id === v.id">
      <button type="button" (click)="open(v)">
        <span class="n"><mat-icon *ngIf="isCustomer && v.progress?.completed; else
num">check_circle</mat-icon><ng-template #num>{{ i + 1 }}</ng-template></span>
        <span class="t"><strong>{{ v.title }}</strong>
          <small>{{ fmtDuration(v.durationSec) }}<span *ngIf="isCustomer && v.progress &&
!v.progress.completed && v.progress.percent > 0"> &middot; {{ v.progress.percent }}% watched</span></small>
        </span>
      </button>
    </li>
  </ol>
</aside>
</div>
</div>

```

components/watch-video/watch-video.component.css (new file)

```

/* Lesson player (Midnight) */
.layout { display: grid; grid-template-columns: minmax(0, 1fr) 340px; gap: 22px; align-items: start; }
.screen { aspect-ratio: 16 / 9; border-radius: var(--radius); overflow: hidden; background: #000; border:
1px solid var(--line); box-shadow: var(--shadow); display: grid; }
.screen video { width: 100%; height: 100%; background: #000; }
.loading-screen { display: grid; place-items: center; padding: 20px; text-align: center; color: var(--
muted); }
.loading-screen.bad { color: var(--danger); }
.info { padding: 16px 2px; display: flex; flex-direction: column; gap: 6px; }
.info h3 { font-size: 22px; margin: 0; }
.muted { margin: 0; color: var(--muted); line-height: 1.55; }
.resume { margin: 0; color: var(--gold-text); font-weight: 600; font-size: 14px; }
.done { display: inline-flex; align-items: center; gap: 6px; color: var(--ok); font-weight: 700; font-size:
14px; }
.empty { padding: 40px 0; text-align: center; color: var(--muted); }
.list { padding: 16px; border-radius: var(--radius); background: var(--surface); border: 1px solid var(--
line); box-shadow: var(--shadow); max-height: 70vh; overflow: auto; }
.list h3 { font-size: 17px; margin: 0; }
.list ol { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; gap: 6px; }
.list li button { width: 100%; display: flex; gap: 12px; align-items: center; text-align: left; padding:
10px; border-radius: 10px; border: 1px solid transparent; background: transparent; color: var(--ink-2);
cursor: pointer; font: 500 14.5px Inter, sans-serif; }
.list li button:hover { background: var(--surface-2); }
.list li.on button { background: var(--gold-soft); border-color: rgba(245, 158, 11, .45); }
.n { flex: none; width: 30px; height: 30px; border-radius: 50%; display: grid; place-items: center;
background: var(--surface-2); font: 700 13px var(--display); color: var(--gold-text); }
.n mat-icon { color: var(--ok); font-size: 24px; width: 24px; height: 24px; }
.t { display: flex; flex-direction: column; min-width: 0; }
.t strong { color: var(--ink); overflow-wrap: anywhere; }

```

```
.t small { color: var(--muted); }
@media (max-width: 900px) { .layout { grid-template-columns: 1fr; } .list { max-height: none; } }
```

components/my-learning/my-learning.component.ts (new file)

```
import { Component, OnInit } from '@angular/core';
import { LearningItem } from '../models/video.model';
import { VideoService } from '../services/video.service';

/** A customer's courses that have videos, with how much is done and a button to continue. */
@Component({
  selector: 'app-my-learning',
  templateUrl: './my-learning.component.html',
  styleUrls: ['./my-learning.component.css']
})
export class MyLearningComponent implements OnInit {
  items: LearningItem[] = [];
  loaded = false;
  loadError = '';

  constructor(private api: VideoService) {}

  ngOnInit(): void {
    this.api.myLearning().subscribe({
      next: list => { this.items = list; this.loaded = true; },
      error: err => {
        this.loaded = true;
        this.loadError = err.status === 404 ? 'Complete your profile first, then your lessons will appear here.' : 'Could not load your lessons. Please try again.';
      }
    });
  }

  trackById(_: number, i: LearningItem): number { return i.courseId; }
}
```

components/my-learning/my-learning.component.html (new file)

```
<div class="page">
  <div class="page-head">
    <div>
      <h2>My learning</h2>
      <p>Pick up your video lessons where you stopped.</p>
    </div>
  </div>

  <div class="form-error" role="alert" id="learningError" *ngIf="loadError"><mat-icon>error_outline</mat-
icon>{{ loadError }}</div>
  <p class="empty" id="noLearning" *ngIf="loaded && !loadError && !items.length">
    None of your courses has video lessons yet. <a routerLink="/customer-view-courses">Browse courses</a>
  </p>

  <div class="grid">
    <article class="card" *ngFor="let i of items; trackBy: trackById" [attr.id]="''learn' + i.courseId">
      <h3>{{ i.courseType }}</h3>
      <p class="muted">{{ i.completedVideos }} of {{ i.totalVideos }} lessons finished</p>
      <div class="bar" role="progressbar" [attr.aria-valuenow]="i.percent" aria-valuemin="0" aria-
valuemax="100">
        <div class="fill" [style.width.%]="i.percent"></div>
      </div>
      <div class="foot">
        <strong class="pct">{{ i.percent }}%</strong>
        <a mat-flat-button color="primary" class="solid continue" [routerLink]="['/watch', i.courseId]"
[queryParams]="i.resumeVideoId ? { video: i.resumeVideoId } : {}">
          {{ i.percent === 100 ? 'Watch again' : i.completedVideos === 0 && i.resumePositionSec === 0 ?
'Start' : 'Continue' }}
        </a>
      </div>
    </article>
  </div>
```

```
    </div>
  </article>
</div>
</div>
```

components/my-learning/my-learning.component.css (new file)

```
/* Customer: my learning (Midnight) */
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 18px; }
.card { display: flex; flex-direction: column; gap: 10px; padding: 20px; border-radius: var(--radius);
background: var(--surface); border: 1px solid var(--line); box-shadow: var(--shadow); }
.card h3 { font-size: 19px; margin: 0; }
.muted { margin: 0; color: var(--muted); font-size: 14px; }
.bar { height: 10px; border-radius: 999px; background: var(--surface-2); border: 1px solid var(--line);
overflow: hidden; }
.fill { height: 100%; background: var(--gold); transition: width .3s ease; }
.foot { display: flex; align-items: center; justify-content: space-between; gap: 10px; margin-top: 4px; }
.pct { font: 800 20px var(--display); color: var(--gold-text); }
.empty { padding: 40px 0; text-align: center; color: var(--muted); }
```